Status	Finished
Started	Friday, 30 October 2026, 2:02 PM
Completed	Friday, 30 October 2026, 2:31 PM
Duration	29 mins 24 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	3
	Inserted line.
		Line one.
		Line two.
		Inserted line.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

# Read inputs dynamically from the standard input stream
input_data = sys.stdin.read().splitlines()

# Ensure we have the required 3 inputs: filename, line number, and content
if len(input_data) >= 3:
    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())
    new_content = input_data[2]

    # 1. Read all lines from the specified input file
    with open(input_filename, 'r') as file:
        lines = file.readlines()

    # Convert 1-indexed line position to 0-indexed list position
    target_index = line_number - 1

    # 2. Ensure the new line ends with a newline character so it shifts other lines down
    if not new_content.endswith('\n'):
        new_content += '\n'

    # 3. Ensure the preceding line ends with a newline character if inserting at the end
    if target_index >= len(lines) and len(lines) > 0:
        if not lines[-1].endswith('\n'):
            lines[-1] += '\n'

```

```
    # 4. Insert the new line at the target position
    lines.insert(target_index, new_content)

    # 5. Write the modified lines directly to the required output file
    with open('output.txt', 'w') as file:
        file.writelines(lines)

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
		Inserted line.
			Line one.
			Line two.
			Inserted line.
			Line three.
			Line four.
				Line one.
				Line two.
				Inserted line.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		4
		Inserted line D.
			Line A.
			Line B.
			Line C.
			Inserted line D.
				Line A.
				Line B.
				Line C.
				Inserted line D.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import sys
import string

# Read the input text from standard input
input_data = sys.stdin.read().strip()

if input_data:
    # Split the input text into individual words
    raw_words = input_data.split()

    word_counts = {}
    for word in raw_words:
        # Convert to lowercase and strip punctuation marks (like ., !)
        cleaned_word = word.lower().strip(string.punctuation)

        # Only count if the word is not empty after stripping punctuation
        if cleaned_word:
            word_counts[cleaned_word] = word_counts.get(cleaned_word, 0) + 1

    # Sort alphabetically by word
    sorted_words = sorted(word_counts.items())

    # Write the clean formatted counts to output.txt
    with open('output.txt', 'w') as file:
        for word, count in sorted_words:
            file.write(f"{word}: {count}\n")

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

# Read inputs dynamically from the standard input stream
input_data = sys.stdin.read().splitlines()

# Ensure we have the required 3 inputs: filename, line number, and new content
if len(input_data) >= 3:
    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())
    new_content = input_data[2]

    # 1. Read all lines from the specified input file
    with open(input_filename, 'r') as file:
        lines = file.readlines()

    # Convert 1-indexed line position to 0-indexed list position
    target_index = line_number - 1

    # 2. Check if the line ends with a newline and format the replacement line
    if 0 <= target_index < len(lines):
        # If replacing an intermediate line, ensure it ends with a newline character
        if not new_content.endswith('\n') and target_index < len(lines) - 1:
            lines[target_index] = new_content + '\n'
        else:
            lines[target_index] = new_content

    # 3. Write the results back out to the required output file
    with open('output.txt', 'w') as file:
        file.writelines(lines)
```

```
30
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`2`
		`Updated line two.`
			`Line one.`
			`Updated line two.`
			`Line three.`
			`Line four.`
				`Line one.`
				`Updated line two.`
				`Line three.`
				`Line four.`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`2`
		`Line B Updated.`
			`Line A.`
			`Line B Updated.`
			`Line C.`
				`Line A.`
				`Line B Updated.`
				`Line C.`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
import sys

# Read inputs dynamically from the standard input stream
input_data = sys.stdin.read().splitlines()

# Ensure we have both the filename and the line number
if len(input_data) >= 2:
    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    # 1. Read all lines from the specified input file
    with open(input_filename, 'r') as file:
        lines = file.readlines()

    # Convert 1-indexed line position to 0-indexed list position
    target_index = line_number - 1

    # 2. Check if the line number exists within the file bounds
    if 0 <= target_index < len(lines):
        # Strip trailing newlines to match the expected format exactly
        print(lines[target_index].rstrip('\r\n'))

```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

# Read inputs dynamically from the standard input stream
input_data = sys.stdin.read().splitlines()

# Ensure we have both the filename and the line number
if len(input_data) >= 2:
    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    # 1. Read all lines from the specified input file
    with open(input_filename, 'r') as file:
        lines = file.readlines()

    # Convert 1-indexed line position to 0-indexed list position
    target_index = line_number - 1

    # 2. Check if the line number exists and remove it
    if 0 <= target_index < len(lines):
        lines.pop(target_index)

    # 3. Write the remaining lines to the required output file
    with open('output.txt', 'w') as file:
        file.writelines(lines)

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	output1.txt
		This is the source file.
		It contains multiple lines of text.
		Here is another line.

Answer: (penalty regime: 0 %)

```
import sys

# Read inputs dynamically from the standard input stream
input_data = sys.stdin.read().splitlines()

# Ensure we have both the source and destination filenames
if len(input_data) >= 2:
    source_filename = input_data[0].strip()
    destination_filename = input_data[1].strip()

    # 1. Read the content from the source file
    with open(source_filename, 'r') as source_file:
        content = source_file.read()

    # 2. Write the content to the destination file
    with open(destination_filename, 'w') as destination_file:
        destination_file.write(content)

```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		output1.txt
			This is the source file.
			It contains multiple lines of text.
			Here is another line.
				This is the source file.
				It contains multiple lines of text.
				Here is another line.
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		output2.txt
			Hello, world!
			Python programming is amazing.
			Let's copy this text to another file.
				Hello, world!
				Python programming is amazing.
				Let's copy this text to another file.
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input3.txt
		output3.txt
			Single line.	Single line.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	3
		Line one.
		Line two.
		eerht eniL.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

# Read inputs dynamically from the standard input stream
input_data = sys.stdin.read().splitlines()

# Ensure we have both the filename and the line number
if len(input_data) >= 2:
    input_filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    # 1. Read all lines from the specified input file
    with open(input_filename, 'r') as file:
        lines = file.readlines()

    # Convert 1-indexed line position to 0-indexed list position
    target_index = line_number - 1

    # 2. Check if the line number exists within the file bounds
    if 0 <= target_index < len(lines):
        original_line = lines[target_index]

        # Determine if the line ends with a newline
        has_newline = original_line.endswith('\n')

        # Remove trailing newline characters for processing
        clean_text = original_line.rstrip('\r\n')

        # Check if the line ends with a period
        has_period = clean_text.endswith('.')
        if has_period:
            clean_text = clean_text[:-1]  # Remove the period temporarily

```

```
        # Reverse the text characters
        reversed_text = clean_text[::-1]

        # Put the period back at the very end of the reversed text
        if has_period:
            reversed_text += '.'

        # Add the newline back if the original line had one
        if has_newline:
            reversed_text += '\n'

        lines[target_index] = reversed_text

    # 3. Write the modified lines to the required output file
    with open('output.txt', 'w') as file:
        file.writelines(lines)

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
filename = input()

with open(filename, 'r') as file:
    text = file.read()

words = text.split()
print("Total words:", len(words))

```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import sys
import string

# Read inputs dynamically from the standard input stream
input_data = sys.stdin.read().splitlines()

if input_data:
    input_filename = input_data[0].strip()

    # 1. Read all words from the specified input file
    with open(input_filename, 'r') as file:
        content = file.read()

    # Split the text into individual words
    raw_words = content.split()

    palindromes = []

    for word in raw_words:
        # Clean the word by removing surrounding punctuation and converting to lowercase
        clean_word = word.lower().strip(string.punctuation)

        # Only process non-empty words that are longer than 1 character
        if clean_word and len(clean_word) > 1:
            # Check if the word reads the same backward as forward
            if clean_word == clean_word[::-1]:
                # Add the original word structure (or clean variant) to your output list
                # Based on your test cases, we keep the original lowercase format
                if clean_word not in palindromes:
                    palindromes.append(clean_word)

    # 2. Write the identified palindrome words to the output file
    with open('output.txt', 'w') as file:
        for palindrome in palindromes:
            file.write(palindrome + '\n')

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import re

filename = input()

with open(filename, 'r') as file:
    text = file.read()

words = re.findall(r'\b[a-zA-Z]+\b', text)

if words:
    longest = max(words, key=len)
    print("Longest word:", longest)

```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.